

Projeto Jaré

Passo a passo anti-procrastinacao para construir o pipeline de leads imobiliarios em Python.

A frase-pipeline (cole no README e no LinkedIn):

*"Pipeline que **ingere** leads de 2 portais imobiliarios via **webhook e API oficial**, **limpa, deduplica e enriquece** os dados, **armazena** em banco, **pontua** a qualidade do lead e **carrega** no Trello e num dashboard, **orquestrado** num servidor."*

Como usar este guia

1. Faça **um passo por vez**. Olhe so o proximo, nunca a lista inteira.
2. Cada passo = **no maximo 5 min** pra começar. Se engrenar, continue.
3. Marque o quadradinho ao terminar. Aquele tique solta dopamina.
4. Travou? Faça a **versao porca** e siga. Refina depois.
5. Faça **commit a cada fase**. Progresso visivel mata a procrastinacao.

63 passos • ~4h de trabalho focado, fatiado em pedacos de no maximo 5 min cada.

FASE 00 Preparar o terreno

~22 min - 8 passos

- ☐ **1** Criar a pasta **Jare/** em Documentos.
1 min So a pasta. Pronto = ela existe. Começar ja e metade do caminho.
- ☐ **2** Abrir o terminal dentro da pasta e rodar **git init**.
2 min Pronto = aparece 'Initialized empty Git repository'.
- ☐ **3** Criar o ambiente virtual: **python -m venv .venv** e ativar.
3 min Ativo = aparece **(.venv)** no inicio da linha do terminal.
- ☐ **4** Criar **requirements.txt** com as libs base.
3 min fastapi, uvicorn, requests, pandas, duckdb, python-dotenv, streamlit, plotly, pydantic. So listar.
- ☐ **5** Instalar tudo: **pip install -r requirements.txt**.
5 min Deixa rodando. Pronto = terminou sem erro vermelho.
- ☐ **6** Criar as pastas **src/**, **data/raw/**, **notebooks/** e um **README.md** vazio.
3 min Esqueleto do projeto montado.
- ☐ **7** Criar **.gitignore** com **.venv/**, **.env** e **data/**.
2 min Evita subir lixo e segredo pro GitHub.
- ☐ **8** Primeiro commit, criar o repo no GitHub e dar push.
3 min Pronto = repo online. Marco psicologico forte.

FASE 01 Garantir os acessos oficiais

~16 min - 4 passos

- ☐ **9** Enviar o email de ativacao do webhook pro suporte do Wimoveis (use **email_wimoveis.txt**).
5 min atendimento@imovelweb.com.br. Sem isso o lead nao chega.
- ☐ **10** Enviar o email pedindo credenciais da API do DFImoveis (use **email_dfimoveis.txt**).
5 min API oficial (HTTP GET + token). Scraping e proibido no termo de uso deles.
- ☐ **11** Ler a doc do payload do Wimoveis e anotar os campos.
3 min LeadName, LeadEmail, LeadTelephone, Message, ExternalId, BusinessType, BrokerEmail, LeadOrigin.

**12**

Montar um lead de exemplo em **data/raw/lead_exemplo.json**.

3 min Pra desenvolver tudo sem depender do suporte responder.

FASE 02 Receber leads do Wimoveis (webhook)

~25 min - 6 passos

**13**

Criar **src/api.py** com FastAPI e a rota **POST /webhook/wimoveis**.

5 min Rodar com **uvicorn src.api:app --reload**.

**14**

Logar no terminal o JSON que chega na rota.

5 min Pronto = voce ve o payload aparecendo.

**15**

Validar o lead com um modelo Pydantic.

5 min Garante que todo lead tem nome, contato e ExternalId.

**16**

Testar enviando o **lead_exemplo.json** com **curl** (ou Thunder Client).

5 min Pronto = retorna 200 e o lead foi logado.

**17**

Salvar cada lead recebido em **data/raw/leads.jsonl**.

3 min Uma linha por lead. Nada se perde.

**18**

Commit: 'webhook wimoveis recebendo leads'.

2 min

FASE 03 Puxar leads do DFImoveis (API)

~22 min - 5 passos

**19**

Criar **src/pull_dfimoveis.py** que faz o GET na API com o token do **.env**.

5 min

**20**

Parsear a resposta JSON virando uma lista de leads.

5 min

**21**

Padronizar pro mesmo formato de campos do Wimoveis.

5 min Mesmos nomes de campo nas duas fontes.

**22**

Marcar **fonte = 'dfimoveis'** e anexar no **leads.jsonl**.

5 min



23

Commit: 'integracao api dfimoveis'.

2 min

FASE 04 Guardar em banco

~22 min - 5 passos



24

Criar **src/db.py** conectando num DuckDB local **data/jare.duckdb**.

5 min DuckDB = SQL local, zero configuracao.



25

Definir o schema da tabela **leads**.

5 min external_id, fonte, nome, email, telefone, mensagem, tipo_negocio, corretor, recebido_em.



26

Funcao **salvar_leads(lista)** que insere os dados.

5 min



27

Carregar o **leads.jsonl** no banco e conferir com um **SELECT count(*)**.

5 min Pronto = o numero bate.



28

Commit: 'persistencia em duckdb'.

2 min

FASE 05 Limpar, deduplicar e enriquecer

~21 min - 5 passos



29

Carregar a tabela num DataFrame pandas em **notebooks/explorar.ipynb**.

3 min



30

Deduplicar pelo **external_id** (o mesmo lead pode chegar 2x).

5 min Um lead nao pode virar dois cards.



31

Normalizar telefone (so digitos, com DDD) e e-mail (minusculo).

5 min



32

Enriquecer: extrair DDD/cidade e criar a flag **tem_contato_valido**.

5 min



33

Salvar a tabela limpa **leads_clean** e dar commit.

3 min

FASE 06 Pontuar os leads (Data Science)

~17 min - 4 passos

- ☐ **34** Definir 3 ou 4 criterios de prioridade do lead.
5 min Tem telefone, mensagem detalhada, tipo de negocio, recencia.
- ☐ **35** Criar **src/score.py** com pontuacao por regras, de 0 a 100.
5 min Comeca simples. Modelo de ML fica pra v2.
- ☐ **36** Adicionar a coluna **score** e ordenar do mais quente pro mais frio.
5 min
- ☐ **37** Commit: 'lead scoring por regras'.
2 min Anote no README que da pra evoluir pra modelo.

FASE 07 Entregar no Trello (cliente)

~35 min - 8 passos

- ☐ **38** Criar o board com as listas: Novos, Em contato, Visita, Proposta, Ganhos.
5 min
- ☐ **39** Gerar a API key e o token do Trello.
5 min Em trello.com/app-key.
- ☐ **40** Salvar as credenciais (Trello e tokens das fontes) no **.env**.
3 min Nunca commitar esse arquivo.
- ☐ **41** Criar **src/trello.py** e fazer 1 card de teste via API.
5 min Pronto = o card aparece no board.
- ☐ **42** Mapear lead em card: nome, contato, mensagem, corretor e o score como label.
5 min Bate exatamente com o pedido do cliente.
- ☐ **43** Criar card so pra lead novo (marcador **jare-ext**: na descricao).
5 min Mesma logica de dedup que voce ja tinha pensado.
- ☐ **44** Ligar tudo: lead chega no webhook e o card nasce no Trello sozinho.
5 min Momento 'uau' pra mostrar pro cliente.



45

Commit: 'integracao com o trello'.

2 min

FASE 08 Dashboard (Data Analytics)

~27 min - 6 passos



46

Criar **app.py** com Streamlit e rodar **streamlit run app.py**.

5 min Pronto = abre sozinho no navegador.



47

Mostrar total de leads, por fonte e por corretor.

5 min



48

Grafico: leads recebidos por dia.

5 min



49

Grafico: leads por tipo de negocio e por faixa de score.

5 min



50

Adicionar filtros por fonte, corretor e periodo.

5 min



51

Commit: 'dashboard em streamlit'.

2 min

FASE 09 Orquestrar e publicar (Data Engineering)

~27 min - 6 passos



52

Criar **run_pipeline.py**: ler novos leads, limpar, pontuar, enviar pro Trello.

5 min



53

Adicionar logging e **try/except** por etapa.

5 min Pra saber exatamente onde quebrou.



54

Subir o FastAPI no seu VPS Hetzner (reaproveita o Caddy/HTTPS ja planejado).

5 min O webhook precisa de URL publica com https.



55

Agendar o **pull_dfimoveis** a cada 5 min (cron no VPS ou Agendador).

5 min DFImoveis e polling.

**56****Testar ponta a ponta: lead de teste, card no Trello, ponto no dashboard.****5 min** Pronto = roda sozinho.**57****Commit: 'pipeline no ar'.****2 min**

FASE 10 Empacotar pro portfolio

~26 min - 6 passos

**58****README: o que e, o problema que resolve, e a frase-pipeline la no topo.****5 min****59****Desenhar o diagrama de arquitetura (Excalidraw ou Mermaid).****5 min** Webhook + API, banco, score, Trello e dashboard.**60****Tirar prints do dashboard e dos cards no Trello pro README.****5 min****61****Listar as skills no README.****3 min** Python, FastAPI, webhooks, integracao de APIs, DuckDB/SQL, pandas, Streamlit, deploy/orquestracao.**62****Publicar o repo e postar o projeto no LinkedIn com a frase-pipeline.****5 min** Recrutador precisa conseguir achar.**63****Commit final e revisar o repo publico pelo celular.****3 min** Ultima conferida. Entregue!